

Lab5：用户程序

南开大学 计算机学院和密码与网络空间安全学院

年级：2023 级

专业：信息安全

姓名及学号：于昊汐（2312642）、陈安琪（2312481）、孙俪菲（2312724）

日期：2025 年 12 月 10 日

一、实验目的

- 了解第一个用户进程创建过程：通过加载 ELF 程序、搭建用户内存空间、配置中断帧，实现从内核态到用户态的首次切换，启动首个用户进程。
- 了解系统调用框架的实现机制：掌握用户态通过ecall触发调用、内核态分发处理、sret返回用户态的完整闭环。
- 了解核心进程管理系统调用：明确sys_fork/exec/wait的协作逻辑，理解进程创建、替换、终止与资源回收的底层实现。

二、实验内容

本次实验我们将完成练习、challenge、gdb调试分支任务，整体概述如下：

练习 0

整合 Lab2（物理内存）、Lab3（中断处理）、Lab4（进程管理）的核心代码，补全对应注释处实现，为后续用户进程创建和系统调用提供基础依赖。

练习 1

补全load_icode函数，完成 ELF 用户程序解析、用户内存空间分配与映射，配置中断帧，实现首个用户进程的加载与执行。

练习 2

补全copy_range函数，实现父进程用户内存空间向子进程的深拷贝，支撑子进程创建时的内存复用。

练习 3

分析系统调用完整流程（触发 - 切换 - 处理 - 返回），梳理fork/exec/wait/exit的协作逻辑与进程状态变迁，理解进程生命周期管理机制。

扩展练习 challenge

基于写时复制（COW）思想优化进程创建，通过共享内存页 + 页错误触发复制的方式，提升子进程创建效率。

GDB 调试任务 1（系统调用以及返回）

调试ecall触发、sret返回过程，观察特权级切换与 CPU 寄存器状态变化，验证硬件层面态切换逻辑。

GDB 调试任务 2（gdb 调试页表查询过程）

加载用户程序符号表，跟踪虚拟地址到物理地址的页表查询（TLB 查找→页表遍历）过程，验证内核页表管理逻辑的正确性。

三、实验过程

练习0：填写已有实验

本实验依赖实验2/3/4。请把你做的实验2/3/4的代码填入本实验中代码有“LAB2”/“LAB3”/“LAB4”的注释相应部分。注意：为了能够正确执行 lab5 的测试应用程序，可能需对已完成的实验2/3/4的代码进行进一步改进。

ANS:

按照以前的代码填充即可，这里注意和之前不一样的地方是我们在trap.c中做出了修改，主要是完善 IRQ_S_TIMER（Supervisor 时钟中断）处理逻辑：

```
/* (1) 设置下次时钟中断 */
clock_set_next_event();

/* (2) ticks 计数器自增 */
ticks++;

/* (3) 每 TICK_NUM 次中断检查当前进程并标记需要重调度 */
if (ticks >= TICK_NUM) {
    ticks = 0;
    if (current != NULL && current != idleproc) {
        current->need_resched = 1;
    }
}
```

上次我们是进行打印，而这次的时钟中断处理主要是为了适配 Lab5 时间片轮转调度，下面是对这部分的一个简单说明：

1. 设置下次时钟中断（clock_set_next_event()）
通过 SBI 接口通知硬件，设置下一次时钟中断的触发时间，每次中断处理完成后必须主动设置下次触发时间，否则后续不会再产生时钟中断，导致调度器无法触发进程切换。
2. ticks 计数器自增
ticks 变量记录系统启动后累计的时钟中断次数。作为时间片的“计量单位”，每触发一次时钟中断，ticks 加 1，累计到 TICK_NUM（100）时，认为当前进程的时间片已用完。
3. 时间片耗尽判断与重调度标记
当 ticks >= TICK_NUM（即累计 100 次时钟中断，对应一个时间片）时：
 - 重置 ticks 为 0，重新开始下一个时间片的计数；
 - 检查当前是否有运行中的用户进程（current != NULL）且不是空闲进程（current != idleproc）；
 - 若满足条件，设置 current->need_resched = 1，标记当前进程需要被重新调度。

Lab5 引入用户进程后，多个用户进程需要通过调度器公平占用 CPU。所以本次修改的核心价值是：

- 提供时间片轮转的触发机制，通过时钟中断定期标记进程需要重调度，确保没有进程长期占用 CPU；
- 兼容用户态与内核态逻辑：仅当当前运行的是用户进程（非 idleproc）时才标记重调度，避免内核空闲时的无效切换；

练习1：加载应用程序并执行（需要编码）

do_execve函数调用load_icode（位于kern/process/proc.c中）来加载并解析一个处于内存中的ELF执行文件格式的应用程序。你需要补充load_icode的第6步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好proc_struct结构中的成员变量trapframe中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的trapframe内容。

请在实验报告中简要说明你的设计实现过程。

请简要描述这个用户态进程被ucore选择占用CPU执行（RUNNING态）到具体执行应用程序第一条指令的整个经过。

ANS:

练习1的核心是完成load_icode函数第6步（用户态陷阱帧trapframe配置），并基于前序实验补充alloc_proc、do_fork等基础函数，最终实现加载ELF格式用户程序并从用户态启动执行。下面我们结合两次代码，说明proc.c的关键实现和流程分析。

1. alloc_proc 函数：进程控制块（PCB）初始化

```
// Lab4代码
proc->state = PROC_UNINIT;           // 初始状态为未初始化
proc->pid = -1;                       // PID 暂设为非法值-1
proc->runs = 0;                      // 运行次数初始为0
proc->kstack = 0;                    // 内核栈地址暂设为0（后续setup_kstack分配）
proc->need_resched = 0;              // 初始不需要调度（ucore用0表示false）
proc->parent = NULL;                 // 父进程初始为NULL
proc->mm = NULL;                    // 内核线程无独立内存管理结构，设为NULL
memset(&proc->context, 0, sizeof(struct context)); // 上下文清零
proc->tf = NULL;                    // 中断帧暂设为NULL（后续copy_thread构造）
proc->pgdir = boot_pgdir_pa;        // 共享内核页表的物理地址
proc->flags = 0;                    // 标志位初始为0
memset(proc->name, 0, PROC_NAME_LEN + 1); // 进程名清零

// LAB5：初始化新增字段
proc->wait_state = 0;               // 等待状态初始化为0
proc->cptr = NULL;                  // 子进程指针
proc->yptr = NULL;                  // 弟弟进程指针
proc->optr = NULL;                  // 哥哥进程指针
```

这里为新进程分配并初始化proc_struct（PCB），确保所有字段（状态、PID、内核栈、内存管理、进程关系等）处于合法初始状态，为后续进程创建和运行打下基础。我们在Lab5中新增了字段，其中wait_state用于标记进程等待状态，cptr/optr/yptr维护进程父子、兄弟关系链表，支持fork/wait等系统调用的进程管理。

2. proc_run 函数：进程切换核心逻辑

这是实现进程上下文切换和页表加载，是调度器选中进程后触发运行的关键函数，在上次实验中已经实现了，这里简单重提一下：

```
bool intr_flag;
struct proc_struct *prev = current;
// 1. 禁用中断,保证切换过程的原子性
local_intr_save(intr_flag);
{
    // 2. 更新当前进程指针
    current = proc;
    // 3. 切换页表,加载新进程的地址空间
    lsatp(proc->pgdir);
    // 4. 执行上下文切换
```

```

        switch_to(&(prev->context), &(proc->context));
    }
    // 5. 恢复中断
    local_intr_restore(intr_flag);

```

核心步骤：

- 禁用中断：避免切换过程中被中断打断，保证原子性；
- 更新 current 指针：将当前运行进程切换为目标进程；
- 加载页表：通过 lsatp 指令更新 satp 寄存器，切换到目标进程的地址空间，用户进程为独立页表，内核线程共享内核页表；
- 上下文切换：通过 switch_to 保存前一个进程的寄存器状态，恢复目标进程的寄存器状态；
- 恢复中断：切换完成后重新启用中断，允许后续中断触发。

3. do_fork 函数：进程创建逻辑

这里补充 Lab5 进程关系的配置，完善进程创建的全流程，包括父进程设置和进程关系链表维护：

```

// 1. 分配并初始化进程控制块
proc = alloc_proc();
if (!proc) {
    goto fork_out; // PCB 分配失败
}

// LAB5: 设置子进程的父进程为当前进程，确保当前进程的wait_state为0
proc->parent = current;
current->wait_state = 0;

// 2. 分配并初始化内核栈
if (setup_kstack(proc) != 0) {
    goto bad_fork_cleanup_proc; // 分配内核栈失败，需要释放 PCB
}

// 3. 根据clone_flags决定是复制还是共享内存管理系统
if (copy_mm(clone_flags, proc) != 0) {
    goto bad_fork_cleanup_kstack; // 复制 mm 失败，需要释放内核栈 + PCB
}

// 4. 设置进程的中断帧和上下文
copy_thread(proc, stack, tf);

// 5. 把设置好的进程加入链表
proc->pid = get_pid(); // 调用get_pid()分配唯一的pid

// LAB5: 使用set_links设置进程关系链接
set_links(proc);
hash_proc(proc); // 插入哈希表hash_list

// 6. 将新的进程设为就绪态
wakeup_proc(proc);

// 7. 返回新线程 pid
ret = proc->pid;
return ret;

```

在上次的实现上我们做出了关键修改：

- 为子进程设置父进程 (`proc->parent = current`)，并重置父进程等待状态 (`current->wait_state = 0`)，支持后续 `wait` 系统调用；
- 使用 `set_links` 函数维护进程链表关系（父子、兄弟），确保 `proc_list` 和哈希表 `hash_list` 正确管理进程。

4. `load_icode` 第 6 步：用户态陷阱帧 (trapframe) 配置

这是我们的核心实现部分，确保进程从内核态切换到用户态后能正确执行用户程序：

```
// 设置用户栈指针：USTACKTOP是用户栈顶地址
tf->gpr.sp = USTACKTOP;

// 设置程序入口点：ELF文件的入口地址
tf->epc = elf->e_entry;

// 设置sstatus寄存器：清除SPP位（设置为用户模式），设置SPIE位（允许中断）
// SPP位为0表示返回用户模式，SPIE位为1表示在用户模式开启中断
tf->status = (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

来逐句解释一下：

- `tf->gpr.sp = USTACKTOP`：
USTACKTOP 是用户栈的最高地址，用户程序的栈从该地址向下生长；
作用：为用户程序设置合法的栈指针，确保用户态代码能正常使用栈空间。
- `tf->epc = elf->e_entry`：
`elf->e_entry` 是 ELF 格式用户程序的入口地址，编译时确定；
作用：`epc` 寄存器记录异常返回后要执行的指令地址，此处设置为用户程序入口，确保切换到用户态后从第一条指令开始执行。
- `tf->status = (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE`：
SSTATUS_SPP：特权级标记位，0 表示用户态 (U 态)，1 表示内核态 (S 态)；清除该位表示返回用户态；
SSTATUS_SPIE：中断使能标记位，1 表示允许中断；用户态需要开启中断以响应时钟中断，支持时间片轮转；
作用：配置 `sstatus` 寄存器，确保返回用户态后权限和中断开关符合预期。

接下来我们来回答练习一中提出的问题：用户态进程从 `RUNNING` 到执行第一条指令的完整流程。

当调度器选中用户进程（状态为 `RUNNING`）后，进程经历内核态上下文切换 → 特权级切换 → 用户态指令执行三个阶段，具体流程如下：

阶段 1：调度器选中进程，触发 `proc_run`

调度器 (`schedule` 函数) 从就绪队列中选中目标用户进程，调用 `proc_run(proc)`；

`proc_run` 禁用中断，更新 `current` 指针为目标进程，通过 `lsatp(proc->pgdir)` 加载用户进程的页表，切换地址空间，确保 CPU 能访问用户程序的代码段、数据段和栈；

调用 `switch_to` 函数，保存当前进程的上下文，恢复目标用户进程的上下文 (`proc->context`)。

阶段 2：上下文切换后，执行 `forkret` 与 `forkrets`

`switch_to` 完成后，CPU 跳转到目标进程的上下文入口 `proc->context.ra` 即 `forkret` 函数；

`forkret` 调用 `forkrets(current->tf)`，传入用户进程的陷阱帧 `tf`，这里就是我们 `load_icode` 第 6 步配置的 `trapframe`；

其中，`forkrets` 函数是汇编实现的特权级切换核心，其作用是：

- 从 `tf` 中恢复用户态寄存器，包括 `sp` 指向用户栈顶 `USTACKTOP`；
- 执行 `sret` 指令，触发从内核态 (S 态) 到用户态 (U 态) 的切换。

阶段 3：`sret` 指令执行，切换到用户态并执行第一条指令

`sret` 指令读取 `tf` 中的配置：

- 将 tf->epc 加载到 pc 寄存器，指向用户程序入口地址；
- 将 tf->status 加载到 sstatus 寄存器，确认用户态权限和中断使能；
- 将 tf->gpr.sp 加载到 sp 寄存器，指向用户栈顶；

特权级切换完成，CPU 进入用户态（U 态），权限受限，不能执行特权指令、不能直接访问内核内存；

pc 寄存器指向 elf->e_entry（用户程序入口），CPU 执行该地址对应的指令，即用户程序的第一条指令。

整理一下流程就是：

调度器选中进程 → proc_run（切换页表+上下文）→ forkret → forkrets（恢复tf寄存器）→ sret指令（特权级切换）→ 执行tf->epc（用户程序入口）

练习2: 父进程复制自己的内存空间给子进程（需要编码）

创建子进程的函数do_fork在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过copy_range函数（位于kern/mm/pmm.c中）实现的，请补充copy_range的实现，确保能够正确执行。

请在实验报告中简要说明你的设计实现过程。

如何设计实现Copy on Write机制？给出概要设计，鼓励给出详细设计。

ANS:

在copy_range 函数中我们补充了注释标记的核心逻辑，具体实现如下：

```
// (1) 获取源页面的内核虚拟地址
uintptr_t src_kvaddr = page2kva(page);
// (2) 获取目标页面的内核虚拟地址
uintptr_t dst_kvaddr = page2kva(npage);
// (3) 复制内存内容，大小为PGSIZE（4KB，页大小）
memcpy((void *)dst_kvaddr, (void *)src_kvaddr, PGSIZE);
// (4) 建立目标页面与线性地址start的映射（继承父进程权限）
ret = page_insert(to, npage, start, perm);
```

我们来完整解释一下这个函数，copy_range 的核心目标是将父进程用户地址空间中 [start, end) 范围内的内存，完整复制到子进程，确保子进程拥有独立的内存空间（写时复制未启用，此处为完整拷贝），实现步骤拆解如下：

首先是前置逻辑：

函数调用链路为 do_fork → copy_mm → dup_mmap → copy_range，进入 copy_range 后：

- 校验地址合法性：确保 start/end 按页对齐（PGSIZE），且属于用户地址空间（USER_ACCESS）；
- 按页遍历：以 PGSIZE 为单位遍历 [start, end) 地址范围，逐个处理父进程的物理页；
- 查找父进程页表项：通过 get_pte(from, start, 0) 获取父进程当前页的页表项（ptep），若不存在则跳过；
- 分配子进程物理页：若父进程当前页有效，则为子进程分配新的物理页（npage = alloc_page()），并获取子进程对应的页表项（nptep = get_pte(to, start, 1)，不存在则创建页表）；
- 继承权限：从父进程页表项中提取用户态权限（perm = *ptep & PTE_USER），确保子进程内存权限与父进程一致。

那么我们新补充的这4步关键操作，完成的是父进程页 → 子进程页的复制与映射：

1. 第一步，获取父进程页面的内核虚拟地址。父进程的物理页需通过内核虚拟地址访问才能读取内容，使用 `page2kva(page)` 宏将 Page 结构体转换为内核虚拟地址，因为内核可直接访问所有物理内存，该地址是父进程页面内容的内核可见入口。
2. 第二步，获取子进程页面的内核虚拟地址。子进程新分配的物理页同样需要内核虚拟地址才能写入内容，通过 `page2kva(npage)` 得到目标地址。
3. 第三步，完整复制页内容。使用 `memcpy` 函数将父进程页面的全部内容从 `src_kvaddr` 复制到 `dst_kvaddr`。此时子进程的物理页与父进程对应页的内容完全一致，包括代码、数据、栈等，但物理地址独立。
4. 第四步，建立子进程页表映射。调用 `page_insert(to, npage, start, perm)` 函数，将子进程的物理页与用户态线性地址建立映射，并设置权限。该操作会更新子进程的页表，确保子进程执行时，访问 `start` 地址能命中新复制的物理页。

最后，就是循环与返回，分两部分：

- 遍历完成：`start += PGSIZE` 切换到下一页，直到 `start` 达到 `end` 或地址溢出；
- 错误处理：若分配页表 / 物理页失败，则返回 `-E_NO_MEM` 表示内存不足；遍历完成后返回 0 表示成功。

这个函数虽然我们补充的比较少，但是它实现了四个关键点，非常重要：

1. 按页复制：内存管理的最小单位是页（`PGSIZE=4KB`），按页遍历确保不遗、漏任何有效内存，且符合页表映射的粒度；
2. 内核态复制：用户态进程无法直接访问其他进程的物理页，通过内核态完成复制是唯一可行方式；
3. 独立物理页：子进程不共享父进程物理页，而是复制一份独立副本，确保写时互不干扰；
4. 权限一致性：继承父进程的用户态权限，确保子进程能正常访问复制后的内存。

我们完成这个函数后，子进程创建时会完整复制父进程的用户态内存空间：

- 子进程执行时，代码段与父进程一致；
- 子进程修改数据 / 栈时，仅修改自身的物理页，不影响父进程；
- 内存权限与父进程一致。

对于练习二提出的设计实现 Copy on Write 机制，我们将在 challenge 中给出完整的设计说明，请耐心等待下阅读哦。

练习3: 阅读分析源代码，理解进程执行 `fork/exec/wait/exit` 的实现，以及系统调用的实现（不需要编码）

请在实验报告中简要说明你对 `fork/exec/wait/exit` 函数的分析。并回答如下问题：

请分析 `fork/exec/wait/exit` 的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？

请给出 `ucore` 中一个用户态进程的执行状态生命周期图（包执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。（字符方式画即可）

ANS:

这一问涉及的函数很多，可能回答会比较繁杂，这里我们用表格的方式来列出关键点。

首先，这四个函数主要关联 `proc.c`、`vmm.c`、`trap.c`、`syscall.c` 四个核心文件，核心函数调用链路 with 代码对应关系如下：

系统调用	用户态封装 (syscall.c)	内核态核心实现 (proc.c)	辅助模块 (vmm.c/trap.c)
fork	sys_fork	do_fork → copy_mm → dup_mmap → copy_range	内存复制依赖 vmm.c 的地址空间管理
exec	sys_exec	do_execve → load_icode	load_icode 依赖 vmm.c 的 mm_map/pgdir_alloc_page 构建新地址空间
wait	sys_wait	do_wait	依赖 proc.c 的进程链表遍历、状态判断
exit	sys_exit	do_exit	依赖 vmm.c 的 exit_mmap 释放内存
系统调用触发	-	-	trap.c 的 exception_handler 捕获 ECALL 指令, 跳转 syscall 分发

接下来我们对四个函数进行分析，来梳理它们的执行流程和执行态区分。

1、fork：创建子进程

函数用于复制父进程的 PCB、内核栈、内存地址空间，生成与父进程独立的子进程，二者从 fork 返回后分别执行。

阶段	执行态	核心操作
触发系统调用	用户态	进程调用 fork(), 通过 ECALL 指令触发陷阱, trap.c 捕获后由 syscall 分发到 sys_fork。
内核态处理	内核态	do_fork 主导, 分配子进程 PCB 并初始化; setup_kstack 分配内核栈; copy_mm 复制父进程内存空间 (页目录、VMA、物理页); copy_thread 复制陷阱帧并设置子进程返回值为 0; 将子进程设为 PROC_RUNNABLE 加入就绪队列; 父进程陷阱帧中设置返回值为子进程 PID。
返回用户态	内核态 → 用户态	父进程通过 sret 切回用户态, 从 fork 返回子进程 PID; 子进程被调度器选中后, 切换上下文并返回 0, 均从 fork 后续指令继续执行。

2、exec：替换进程内存空间

函数用于销毁当前进程的用户态内存空间，加载 ELF 格式新程序，重置执行上下文，让进程执行新程序，PID 保持不变。

阶段	执行态	核心操作
触发系统调用	用户态	进程传入新程序路径、ELF 数据等参数，调用 exec 相关接口，ECALL 触发陷阱后分发到 sys_exec。
内核态处理	内核态	do_execve 释放当前进程旧内存；load_icode 解析 ELF 头部，为代码段、数据段、BSS 段和用户栈分配内存并建立映射；重置陷阱帧，sp=USTACKTOP 设用户栈顶、epc=elf->e_entry 设程序入口、status 设为用户态允许中断。
返回用户态	内核态 → 用户态	sret 指令直接跳转到新程序入口地址，原程序因内存已释放不再执行，无传统意义上的返回。

3、exit：进程终止

函数用于进程主动终止，释放用户态资源，通知父进程，进入僵尸态等待回收，避免资源泄漏。

阶段	执行态	核心操作
触发系统调用	用户态	进程传入退出状态码，调用 exit()，ECALL 触发陷阱后分发到 sys_exit。
内核态处理	内核态	do_exit 释放用户态内存；设置进程状态为 PROC_ZOMBIE，保存退出状态到 exit_code；唤醒处于 WT_CHILD 状态的父进程，将当前进程的子进程托付给 initproc；触发调度切换到其他就绪进程，当前进程不再执行。
后续处理	内核态	僵尸态进程仅保留 PCB 中的退出状态和进程关系，其余资源已释放，等待父进程通过 wait 回收。

4、wait：父进程回收子进程

函数用于父进程阻塞等待子进程终止，回收子进程的退出状态和 PCB 资源，消除僵尸进程。

阶段	执行态	核心操作
触发系统调用	用户态	父进程传入目标子进程 PID 和状态存储指针，调用 wait()，ECALL 触发陷阱后分发到 sys_wait。
内核态处理	内核态	do_wait 遍历子进程链表：若找到 PROC_ZOMBIE 子进程，将退出状态写入用户态指针，回收其内核栈和 PCB；若无僵尸子进程但存在子进程，父进程设为 PROC_SLEEPING 阻塞；子进程 exit 后父进程被唤醒，重新执行遍历逻辑；若无子进程则返回错误。
返回用户态	内核态 → 用户态	子进程回收完成后，sret 切回用户态，父进程从 wait 返回子进程 PID，退出状态已写入指定指针。

对于后两个问题：

内核态与用户态程序是如何交错执行的？

主要遵循下面的流程：

1. 用户态进程执行 ECALL 指令，触发 USER_ECALL 异常，CPU 自动切换到内核态，保存用户态上下文（寄存器、程序计数器等）到陷阱帧 tf；
2. 内核态通过 trap.c 的 trap_dispatch 分发到 exception_handler，最终调用 syscall 函数根据系统调用号（tf->gpr.a0）匹配 sys_fork/sys_exec 等处理函数；
3. 内核态完成资源管理、状态修改等核心工作后，更新陷阱帧中的关键信息；
4. 执行 sret 指令，CPU 恢复用户态上下文，跳转到 tf->epc 指向的地址，回到用户态继续执行。

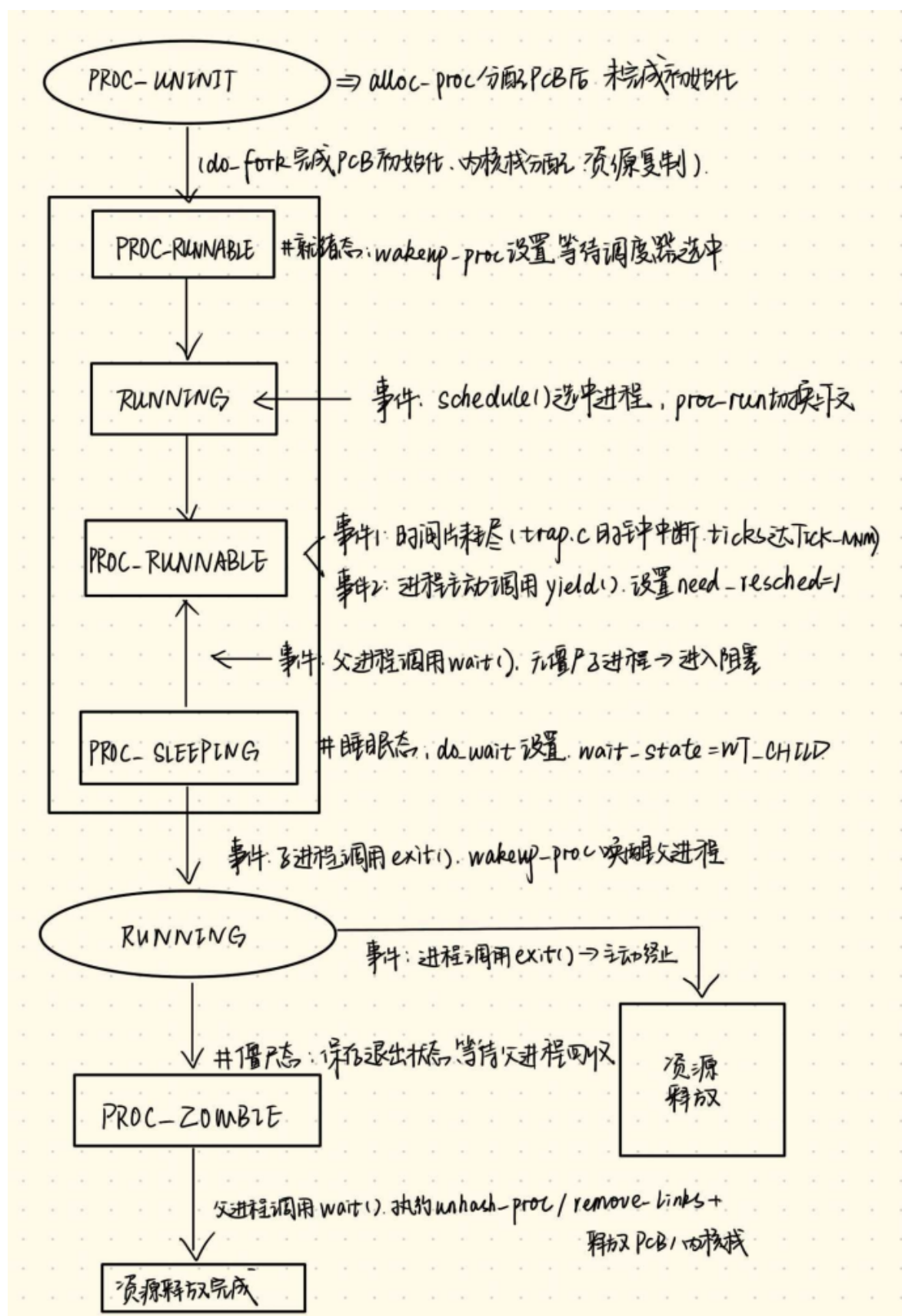
例外情况：exit 触发后内核态直接切换进程，不返回用户态；exec 返回用户态后直接执行新程序，原逻辑终止。

内核态执行结果是如何返回给用户程序的？

分情况：

1. 普通返回值（如 fork 的 PID、getpid 的 PID）：内核态将结果写入陷阱帧的 gpr.a0 寄存器，用户态从 a0 读取；
2. 退出状态（exit → wait）：exit 时内核将状态保存在子进程 PCB 的 exit_code，wait 时通过合法用户态指针写入，写入前需经 user_mem_check 校验地址合法性；
3. 程序入口与栈指针（exec）：内核态直接修改陷阱帧的 epc（程序入口）和 gpr.sp（用户栈顶），sret 后直接生效，确保新程序能正常启动。

下面我们给出ucore中一个用户态进程的执行状态生命周期图：

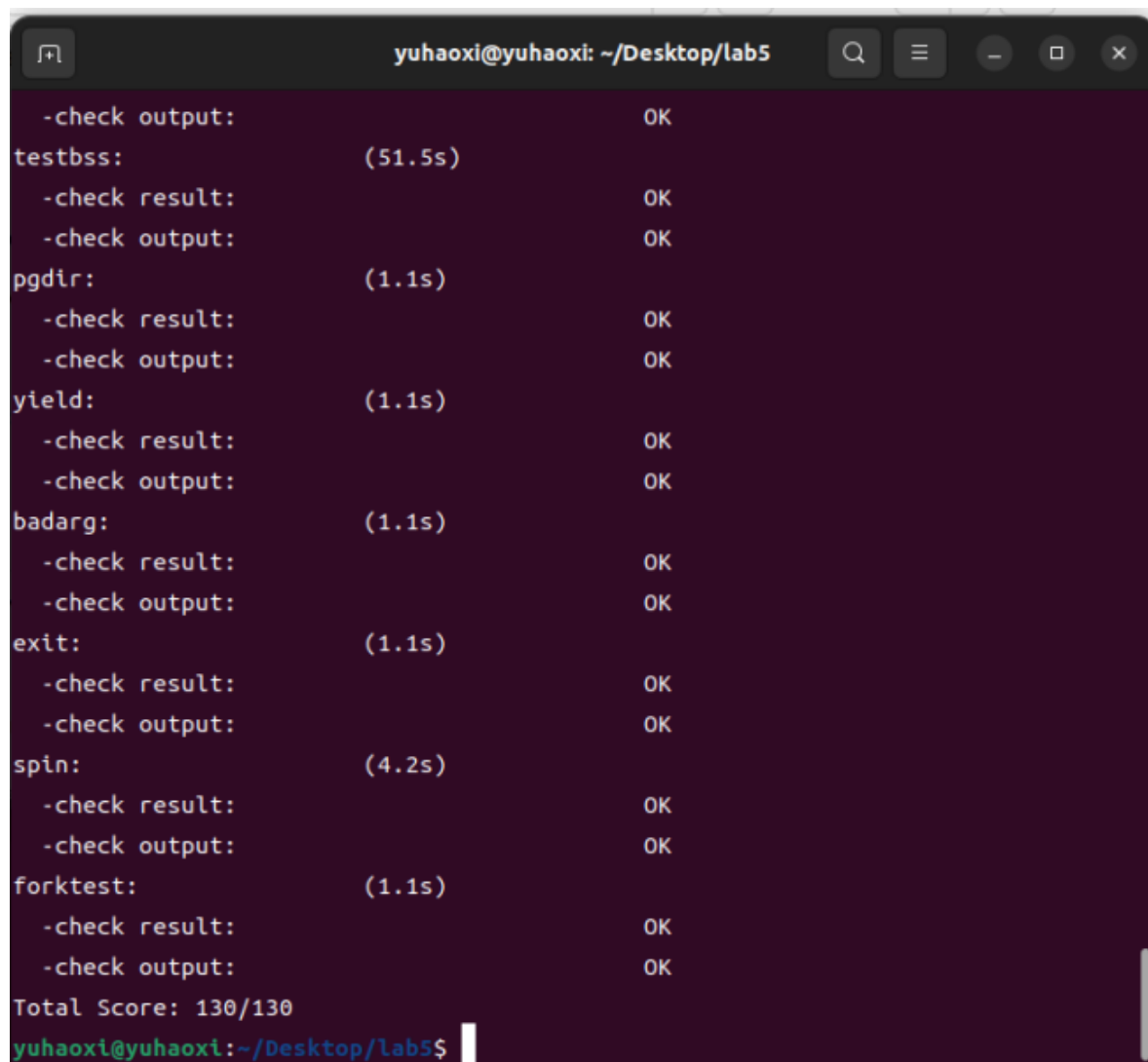


图中的核心状态变换:

- 未初始化→就绪: do_fork 完成所有初始化工作后, wakeup_proc 触发;
- 就绪→运行: 调度器选中进程, proc_run 完成上下文切换和页表加载;
- 运行→就绪: 时间片耗尽或主动放弃 CPU (yield);
- 运行→睡眠: 父进程 wait 时无僵尸子进程, 主动阻塞;
- 睡眠→就绪: 子进程 exit 唤醒父进程;
- 运行→僵尸: 进程调用 exit, 释放核心资源后进入等待回收状态;
- 僵尸→资源释放: 父进程 wait 回收 PCB 和内核栈, 进程生命周期结束。

所以，fork/exec/wait/exit 的核心能力均由内核态实现，用户态仅承担 触发调用和接收结果轻量角色。内核态通过陷阱机制接管控制权，依托 PCB 管理进程状态、通过 vmm 模块管理内存资源、通过调度器实现进程切换，最终通过修改陷阱帧将执行结果反馈给用户态。进程状态的转换严格遵循 proc.c 定义的规则，确保进程创建、执行、终止、回收的全流程有序且无资源泄漏。

最后，我们make grade一下：



```
yuhaoxi@yuhaoxi: ~/Desktop/lab5
- check output: OK
testbss: (51.5s)
- check result: OK
- check output: OK
pgdir: (1.1s)
- check result: OK
- check output: OK
yield: (1.1s)
- check result: OK
- check output: OK
badarg: (1.1s)
- check result: OK
- check output: OK
exit: (1.1s)
- check result: OK
- check output: OK
spin: (4.2s)
- check result: OK
- check output: OK
forktest: (1.1s)
- check result: OK
- check output: OK
Total Score: 130/130
yuhaoxi@yuhaoxi: ~/Desktop/lab5$
```

可以看到，所显示的应用程序检测都输出ok，分数为130/130，实现基本正确。

扩展练习 Challenge

1、实现 Copy on Write (COW) 机制

给出实现源码、测试用例和设计报告（包括在cow情况下的各种状态转换（类似有限状态自动机）的说明）。

这个扩展练习涉及到本实验和上一个实验“虚拟内存管理”。在ucore操作系统中，当一个用户父进程创建自己的子进程时，父进程会把其申请的用户空间设置为只读，子进程可共享父进程占用的用户内存空间中的页面（这就是一个共享的资源）。当其中任何一个进程修改此用户内存空间中的某页面时，ucore会通过page fault异常获知该操作，并完成拷贝内存页面，使得两个进程都有各自的内存页面。这样一个进程所做的修改不会被另外一个进程可见了。请在ucore中实现这样的COW机制。

由于COW实现比较复杂，容易引入bug，请参考 <https://dirtycow.ninja/> 看看能否在ucore的COW实现中模拟这个错误和解决方案。需要有解释。

这是一个big challenge.

2、说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

对于Challenge的回答我们单独写了一个报告，请您务必阅读同文件夹下的“Challenge.pdf”，万分感谢！

分支任务

我们完成了Lab2和Lab5的分支任务，并单独撰写了报告，请您务必阅读同文件夹下的“分支任务.pdf”，万分感谢！

四、讨论

实验知识点与OS原理知识点对应关系

实验知识点	OS原理知识点	理解与关联
用户进程创建（ELF解析、内存空间搭建、trapframe配置）	进程创建机制、程序加载原理	实验通过load_icode解析ELF文件，为用户程序分配独立内存空间并配置陷阱帧，对应OS中“为进程分配资源、建立执行环境”的核心流程，是将静态程序转化为动态进程的具体实现。
系统调用框架（ecall触发、内核分发、sret返回）	特权级隔离、系统调用机制	实验中用户态通过ecall触发异常切换到内核态，内核通过syscall函数分发处理，最终用sret返回，完美对应OS中“用户态-内核态安全切换”的设计，解决了用户程序权限受限的问题。
fork/exec/wait/exit系统调用实现	进程生命周期管理、资源回收机制	fork通过复制父进程资源创建子进程，exec替换进程内存空间，exit释放资源并进入僵尸态，wait回收子进程PCB，这与OS中“进程创建-执行-终止-回收”的完整生命周期管理逻辑一致，确保进程资源不泄漏。
内存复制（copy_range）与COW思想	内存管理、写时复制优化	copy_range实现父进程内存深拷贝，保证子进程独立性；COW通过“共享+写时拷贝”优化，减少创建子进程的内存开销，对应OS中“内存高效复用”的设计思路，平衡了独立性与资源利用率。
GDB调试页表查询、TLB查找	虚拟内存地址转换、MMU工作原理	实验通过调试QEMU模拟的TLB查找和页表遍历过程，直观呈现了OS中虚拟地址到物理地址的转换流程，验证了“TLB缓存提速、页表多级索引”的硬件模拟逻辑。

OS原理中重要但实验未覆盖的知识点

- 进程间通信（IPC）机制：**OS原理中管道、消息队列、共享内存等IPC方式是进程协作的关键，但实验仅聚焦进程生命周期管理，未涉及进程间数据交换与同步的实现。
- 文件系统：**真实OS中用户程序通过文件系统访问外部存储，但实验中用户程序预先编译到镜像中，未涉及文件的创建、读写、删除等文件系统核心功能。

3. **死锁检测与避免**：OS原理中死锁是多进程资源竞争的重要问题，需通过资源分配策略、银行家算法等规避，但实验中进程数量少、资源竞争简单，未涉及死锁相关机制。
4. **内存换页（Swap）机制**：真实OS会将部分内存数据换入换出磁盘以扩展可用内存，但实验中内存资源固定，未实现虚拟内存的换页功能，仅涉及内存分配与映射。
5. **线程与线程调度**：OS原理中线程作为轻量级进程，共享进程资源并支持更细粒度的调度，但实验仅实现进程级调度，未涉及线程的创建、管理与调度逻辑。

五、实验结论及心得体会

实验结论

1. 成功实现了首个用户进程的创建与执行：通过整合Lab2-4的核心代码，补全load_icode函数，完成了ELF文件解析、用户内存空间搭建和陷阱帧配置，实现了从内核态到用户态的首次切换，确保用户程序能从指定入口地址正常执行。
2. 完善了系统调用框架与进程管理机制：实现了ecall触发、内核分发、sret返回的系统调用闭环，完成了fork/exec/wait/exit的协作逻辑，能够正确处理进程创建、替换、终止与资源回收，通过make grade测试，所有应用程序检测均输出OK，得分为130/130，验证了实现的正确性。
3. 理解了虚拟内存与特权级切换的底层逻辑：通过调试页表查询和系统调用流程，明确了虚拟地址到物理地址的转换过程，以及用户态与内核态的安全切换机制，验证了MMU、TLB等硬件模拟逻辑的合理性。

心得体会

1. 本次实验让我们对操作系统的“分层设计”有了深刻认识：用户程序运行在受限的用户态，通过系统调用接口获取内核服务，内核态负责资源管理和特权操作，这种隔离设计既保证了系统安全，又为用户程序提供了灵活的服务，让我理解了“为什么用户程序不能直接访问硬件”的核心原因。
2. 编程实现过程中，细节处理至关重要：例如load_icode中陷阱帧的sp、epc、status寄存器配置，若稍有偏差就会导致用户程序无法启动；copy_range函数中按页复制和权限继承的逻辑，直接影响子进程内存的独立性，这让我们体会到OS底层代码“差之毫厘，谬以千里”的严谨性。
3. GDB调试成为理解底层逻辑的关键工具：通过双重GDB调试QEMU和ucore，我们直观看到了ecall和sret指令的处理流程，以及TLB查找、页表遍历的具体过程，将抽象的“地址转换”“特权级切换”概念转化为可跟踪的代码执行流程，极大提升了对OS底层机制的理解。
4. 对“优化思想”有了更实际的感悟：从copy_range的深拷贝到COW的“写时复制”，实验展现了OS设计中“平衡性能与资源开销”的思路，让我们明白优秀的OS不仅要实现功能，还要考虑资源利用率的优化，这种设计思维对后续编程实践具有重要启发。
5. 团队协作提升了实验效率：实验涉及代码整合、功能实现、调试验证等多个环节，通过分工合作，我们快速定位并解决了时钟中断适配、陷阱帧配置错误等问题，深刻体会到复杂系统开发中团队协作的重要性。